Final Design Report

Version 3

Date: 2025-04-08

Course: ECSE 211

Designers:

- Hamza Rtel Bennani
- Mohamed-Amine Benzaid
- Yutong Zhang
- Williams Lendjoungou
- Justin Cyr

Professors:

- Prof. Benoit Boulet
- Prof. AJung Moon
- Prof. Roufaida Bensalem

Table of Contents

1. Introduction *1*

Team Roles: **1**
Team Contract Feedback **1**
Team Availability & Schedule **2**
Project Milestones and Management **2**
Time and Budget Adjustments **3**

2. Specifications *3*

Problem Statement **4**
Requirements: **4**
Fire detection & extinguishing 4
Navigation 4
Scanning 4

Constraints: **5**
Adjusted specifications **5**
Assumptions **5**

3. Design *5*

Overview of System Design **6**
Robot Overview 6
System Architecture 6
System Workflow 7

Evolution from Concept to Final System **7**
Hardware Design **9**
Displacement: 9
Sensors: 9
Dispenser: 10
Battery Holder: 12
Speaker/ Siren: 12
Brick Pi: 12

Software Design: **13**
Integration of Hardware and software **15**
Recreating the System **14**
Design Modifications to Avoid **15**

4. Testing *15*

Dispenser Test **15**
Sensors Test **17**
Color sensor testing 17
Detection software testing 19

Ultrasonic sensor testing 20
Navigation Test: **20**
Speaker Test: **22**
Emergency Button Test: **22**
5. Performance ***23***
Performance **23**
Limitations **24**
6.Appendix ***26***
Appendix A: Gantt Chart **26**
Appendix B: TimeSheet **28**
Appendix C: Color Sensor Data Table **30**
Appendix D: Color data analysis graphs **31**
Appendix E: Ultrasonic Sensor Data **32**
Appendix F: Designs Photos and Sketches **32**

1. Introduction

Team Roles:

Justin Cyr: Project Manager
Yutong Zhang: Testing Lead
Hamza Rtel Bennani: Hardware Lead
Williams Lendjengou: Documentation
Mohamed-Amine Benzaid: Software Lead

Team Contract Feedback

Guiding principles
We followed these values:
Performance, Equality, Inclusivity, Respect.

Preferred means of communication:
WhatsApp, Discord

Accountability work in our team:
Decisions were made as a group, with every member encouraged to share their opinion. We appointed a time manager to ensure that we stayed on track with deadlines. During meetings, someone was always assigned to take notes to help keep a record of discussions and decisions.

Conflict resolution
We committed to open communication, listening to everyone's opinion, and involving all members in problem-solving.

Team contract signature:
Justin Cyr
Yutong Zhang
Williams Lendjoungou
Mohamed-Amine Benzaid
Hamza Rtel Bennani

Following this contract helped us maintain good teamwork throughout the robot design project. Our communication system played a key role in improving productivity. The project manager consistently reminded the group of upcoming deadlines through WhatsApp messages, typically sent three days in advance. However, some issues arose when not all members saw the messages on time. To improve this, our team contract could include a response-time rule to ensure accountability. Additionally, the contract could have been more detailed, specifying who was responsible for what, and how many hours each team member should contribute weekly. These changes could have reduced confusion and made communication even smoother.

Team Availability & Schedule

Each team member was expected to dedicate approximately 9 hours per week to the project, including lab time, meetings, and individual work. Everyone was available on Wednesdays from 14:30 to 16:00, which became our consistent weekly meeting time. Our senior engineer also joined these meetings between 15:00 and 16:00 to discuss our progress and offer suggestions. Each member spent around 9 hours per week on average. When someone fell short in a week, those hours were made up in the following one. Given our differing schedules, this flexible approach helped ensure fairness while still meeting project goals.

Table 1. Lab Schedule & Team Capabilities

Team Members	**Availability** (for the final project without DPM lecture)	**Capabilities**
Justin Cyr	Monday: 11:35-5:00, Wednesday 11:35-5:00, Thursday: 12:30-13:30	Arduino, hardware, strong in drafting reports, basic Python software skills
Yutong Zhang	Monday: 11:35-5:00, Tuesday: 12:00-15:00, Wednesday: 14:30-17:00	Software, programming, mechanical knowledge
Hamza Rtel Bennani	Monday: 11:35-5:00, Tuesday: 11:35-5:00, Wednesday: 12:00-16:00	Strong software skills, CAD (NX and SolidWorks)
Mohamed-Amine Benzaid	Monday: 15:00-17:00, Wednesday: 14:00-16:00, Thursday: 13:00-19:00	Software-focused (App/Web dev, Algorithms, Full Stack, Python))
Williams Lendjengou	Monday: 15:00-17:00, Tuesday: 14:00-19:00, Wednesday: 13:00-19:00	Programming (Python, Java, Bash, C, HTML)

The diversity in skills among team members greatly benefited our project. However, the coding work was particularly demanding, and Mohamed-Amine and Williams—our most experienced programmers—took on a larger workload during the design phase. Having strong programming expertise on the team proved essential to meeting project goals.

Project Milestones and Management

We identified three major milestones:

1. Project Planning, including definition of client needs.

2. Design Review, presenting our proposed solution.
3. Final Demo, showcasing the final working robot.

We tracked our progress using a **Gantt chart** and a **time budget excel sheet**, both of which are included in **Appendices A and B**. Each task in the Gantt chart was associated with specific hours and assigned team members. However, we encountered some unexpected delays that required us to adjust both the chart and the time allocations.
For instance, coding took significantly more time than anticipated and should have started earlier in the process. Additionally, our team tended to work more intensively just before submission deadlines, which sometimes created time pressure. In the first week, we only logged about 25 total hours as a group due to procrastination. This led us to increase our effort later in the schedule to catch up and ensure that we met client expectations.

Time and Budget Adjustments

Our Gantt chart outlines the major milestones and weekly deliverables that structured our design process. Key project phases included:

-Project Planning and Research (Week 1)
-User Requirements & Validation (Week 1)
-System & Hardware Modelling (Week 2)
-Software Development and Testing (Week 3)
-Design Review Preparation and Iteration (March 24th)
-Final Demo Preparation (Week 4)

Each phase was given a specific timeline with targeted deliverables and team member responsibilities. We realized that it was easier to split work in terms of week instead of days. The most time-consuming portion was software development, which extended into Weeks 3 and 4 and required reallocation of hours to those most experienced in coding.
Our time budget was equally distributed, with each member contributing to 36 hours to reach a total of 180 hours. Work was split across meetings, individual tasks, and testing phases. However, due to delay, such as hardware malfunction on March 28th, we had to revise our Gantt chart timeline. The slack days built into the final demo phase allowed us to catch up effectively and make final adjustments. Team hours were transparently tracked and reported during weekly meetings. Everyone honestly declared their time and input, leading to a fair and efficient work environment. A shared Excel sheet was used to log hours per task and week, ensuring alignment with Gantt chart schedule and workload distribution.

2. Specifications

Problem Statement

Our team has been tasked with designing a Firefighter Rescue Robot. The robot must autonomously navigate a simulated building environment, identify and extinguish two fire hazards represented by 2.54 cm^2 red stickers, and return to its base of operation after. Extinguishing is simulated by dropping a 2.54 cm^3 yellow/orange foam cube onto the fire.
The robot must avoid obstacles, walls, and other elements such as furniture represented by green stickers. It must only enter rooms with active fires by the main door. This door is represented by an orange line entrance. The robot must complete its mission within 3 minutes. Additionally, the robot should play a fire truck siren sound at the beginning of the mission and stop when a fire room is reached. A manual emergency stop mechanism must also be included to stop all movement immediately when pressed. This problem requires a multidisciplinary design solution involving sensor integration, autonomous navigation, time efficiency, hardware implementation, and clear interpretation of the task under fixed constraints using mostly Brick Pi kits only.

Requirements:

<u>Fire detection & extinguishing</u>

The robot must detect the red sticker (2.54 square meter) using color sensor.
It must release the sandbag (yellow cube) upon detection to extinguish the fire.
It will continue to search for additional fire after the first detection.

<u>Navigation</u>

1. The robot must use an ultrasonic sensor to detect and avoid walls and obstacles.
2. It must not collide with walls and obstacles while moving.
3. It should only enter the room with fires (red cubes).
4. It must enter and leave the room through the main entrance (22 cm-wide).
5. It must play a siren sound when it starts, stopping when it reaches the fire room.
6. It must include a manual emergency stop button (touch sensor) that halts all movements immediately.
7. It must start at the fire station (0,0) and return to it after completing the task.

<u>Scanning</u>

1. The color sensor should have a detection range of at least 2 cm to ensure early fire recognition.
2. The fire dispensing system should be less than 10 cm in all dimensions to allow space for navigation components.
3. The sandbag dropping mechanism should function within 0.3 seconds after fire detection to optimize mission time.
4. The ultrasonic sensors must detect walls within a 4 cm range to allow safe stopping distance.

Constraints:

1. Must only use provided LEGO & Brick Pi in lab kits.
2. No 3D printing is allowed.
3. Only approved additional materials may be used (tape, paper).
4. The test environment is fixed at 1.2m X 1.2m
5. The rooms with fire(s) should be the only ones that the robot enters.
6. Obstacles and walls must be avoided.
7. Other colored objects representing furniture (green stickers) and humans (blue cubes) must not be hit.
8. The robot should be less than 22 cm wide to enter the room.

Adjusted specifications

1. Color sensor needs to be stable and close to the ground with distance 2 cm.
2. Color sensor should always be slightly tilted with an angle of 30 degrees to the vertical direction.
3. The turning speed for the robot should be limited to ensure timely detection of color
4. The power source of the robot should be fully charged before starting the mission for optimal functioning of sensors
5. The robot will not stop when detecting unknown objects (colors)

Assumptions

-The red sticker is always located on the floor and lies flat.
-There will at least two fires placed in room #2.
-The robot's maximum width must be slightly less than 22 to pass the door.
-The emplacement of room #2 remains the same and the room sections does not change.
-The test environment is indoors with stable lighting conditions and the stickers remains the exact same color.
-The adherence of the surface is constant.
-The robot does not need to re-extinguish fires already marked as handled and a fire is removed once it is extinguished.
-The foam cubes used as fire suppressants can be dropped freely without a precise aiming system as long as it touches the fire.
-The rooms color stays the same.

3. Design

Overview of System Design

Robot Overview

We followed the V-cycle approach, beginning with client requirements and concluding with a fully functional product. Ultimately, the robot is capable of navigating from its base to Room #2 and returning. It can scan the environment, detect fire, and navigate while avoiding obstacles:

- Fire detection: When a fire is detected, the robot stops and drops an orange/yellow cube.
- Furniture detection: If furniture is detected, the robot goes back and then turns left at a random angle.
- Wall detection: The ultrasonic sensor detects walls, and the robot turns left at a random angle.
- Other room detection: If the robot detects a room that is not the target chamber, it also turns to comeback into room #2.
- After extinguishing 2 fires, the robot comeback to base.

System Architecture

The system consists of a BrickPi as the central controller, interfacing with multiple sensors and motors:

- Two motorized wheels enable movement.
- One additional motor control a dispenser for dropping cubes.
- A color sensor detects fires and differentiates between furniture and other rooms.
- An ultrasonic sensor scans walls for navigation and obstacle detection.
- Other Lego parts stabilize and support the robot

Table 2. Components and Interactions

Component	**Function**
BrickPi	Central controller that manages motors and sensors.
Lego pieces	Structural support to hold components together.
2 large motors	Control the movement of the robot.
1 medium motor	Controls the dispenser.
1 ultrasonic sensor	Scans for walls and obstacles.

Component	Function
1 color sensor	Detects fire, furniture, and other rooms.
1 battery	Powers the system.
6 cables	Connect various components.
2 steering balls	Improve stability and movement.

Input: Color sensor, Ultrasonic sensor
Output: 3 motors (2 for the wheels and 1 for the dispenser)

System Workflow

Initialization:

- The Brick Pi powers up, initializing all sensors and motors.
- The robot starts at its base station and enter room #2 using and hard code path.

Navigation Towards the Fire into room #2:

- The ultrasonic sensor continuously scans for walls.
- The color sensor scans the environment for fire, furniture, and other rooms.
- The motors drive the wheels forward in 3 cm increments then stops to scan.
- After each movement, the robot scans from left to right:
    - If no obstacle is encountered, the robot continues moving.
    - If a wall or object is detected, the robot turns to avoid it.
    - If fire is detected, the robot stops and activates the dispenser motor to drop an orange cube.

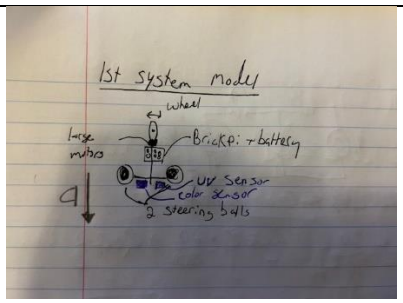
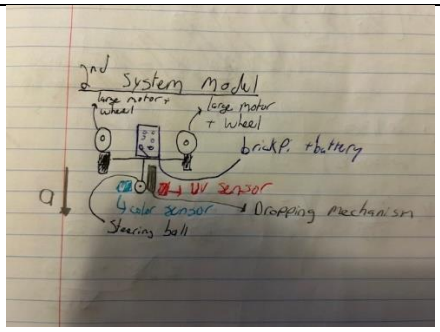
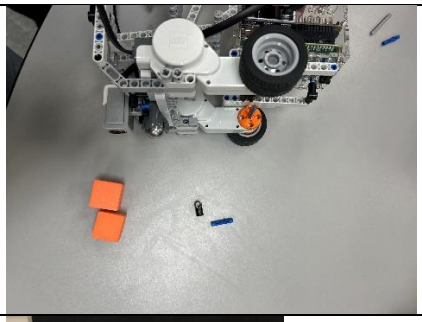
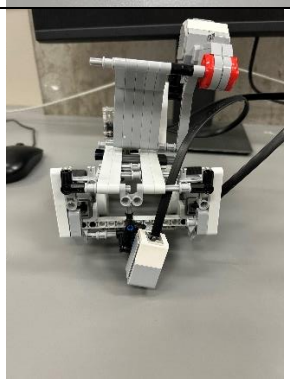
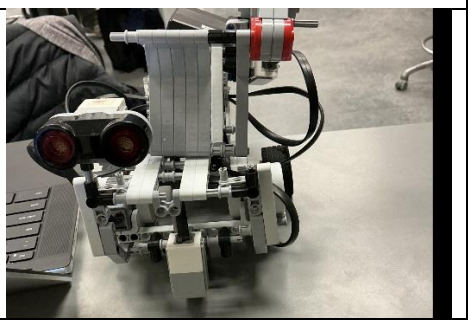
Return to base

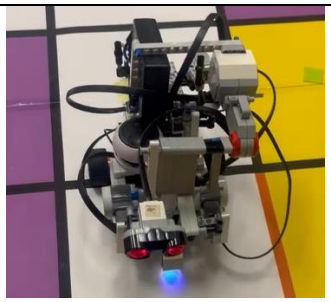
- After extinguishing two fires, the robot returns to base by moving backward retracing it steps

Evolution from Concept to Final System

Table 3. System Model and Design Iterations

Stage	Key Features	Challenges Encountered	Modifications	Scheme and Images

System model 1	-1 motor for a wheel -1 motor for a dispenser -2 steering balls	Not able to move forward and turn with only one motor	Change the navigation system model	
System model 2	-2 medium motors for 2 wheels -1 large motor for a dispenser -1 steering ball -1 color sensor	Make turns with two rotating back wheels	Adjust the code with a function that make the robot turn	
System design 1	-Chassis -Robot base -Wheels	Stability and implementation of other subsystems	Adjust the hardware	
System design 2	-Added dispenser system -Added color sensor	Dropping the cube precisely on the target	Adjust the hardware and code	
System design 3	-Added an ultrasonic sensor	Precision scanning the code Problem while navigating, the robot seems unbalanced, when moving forward it deflects to the left	Adjust the code, modify the hardware to be more stable, adjust the weight with the position of the battery	

Final design	-Added a steering ball -Added a battery holder -Added speaker -Added emergency button	-	-	

Some details about subsystems, ideas and iterations will be tackle more in detail in the hardware design part.

Hardware Design

Displacement:

To displace the robot accuracy around the simulation, a set of two movements are required: Rotation and Translation. This is why system model 2 was better for our project.

This is why we decided to use the two wheels with a steering ball design, allowing a rotation without changing the coordinates of the robot. Each wheel has a different motor. With this set, when a wheel moves forward and other moves backward, the robot turns. The second steering ball was added at the back of the robot to support the newly added weight of the battery. [Appendix F: Figure 1]

The robot only moved correctly and straightforward when all the weight was balanced on each side.

Sensors:

Touch Sensor (emergency button): [Appendix F: Figure 3]

- We opted for an easy to implement emergency button, represented by a touch sensor on the side of the Brick Pi. However, due to budget restrictions (not enough tine left), the emergency button is not working during hardcoded path, it only works during the scanning and backtracking phases

Color Sensor: [Appendix F: Figure 2]

- As documented by our testing lead, the color sensor would need to be close to the ground (about 2 centimeters) facing down. The color sensor is placed in front of the robot to make sure to scan obstacles and fire before running above them.
- During multiple tests, we understood that on the simulation terrain and its brightness, the color sensor would give better results if tilted a bit forward. Indeed, if facing completely down the sensor would not have enough light to give accurate RGNB values.

Ultra Sonic Sensor (US sensor): [Appendix F: Figure 2]

- The pertinence of a US sensor was questioned by us during the whole design process. At the beginning, we thought that we could have a "map" of room #2 area implemented in the software. The robot could have known at any time where he was positioned and facing in the room. With this initial idea, a US sensor would not have been relevant cause the program would adjust based on the position. However, this idea has been compromised by two elements: the hardware unreliability (imprecise movements) and an easier software design (explained later).
- Following these events, we decided to add a US sensor at the front of the robot, on top of a motor (right one). At this position, the robot can scan walls 2cm away from them and adjust is position after.

Dispenser:

We determined several requirements for the dispenser subsystem.

- Being able to store at least 2 cubes of 2-inch side. One for each red sticker required to extinguish.
- Being able to drop one cube on at least two red stickers.
- The dispenser should drop accurately the cubes over the red stickers. As answered by our senior engineer, only touching a part of the red sticker with the cube would be enough to extinguish it.

Multiple design ideas have been explored and thought about by our design team. Here an overview of the designs explored.

A. Robotic Arm [Appendix F: Figure 4]

At a first glance we imagined a futuristic robotic arm for our robot. This arm could hold the cube from a cube inventory to release them on the fires. This method has been discussed but quickly dismissed as it would have been difficult to build with the given material, and not stable due to its weight and great variety of movements at a high distance from the ground. Thus, the robotic arm has not been built.

B. Mechanic Treadmill [Appendix F: Figure 5 and 6]

Our next idea was a mechanical treadmill that would rotate until the cubes placed on top of it would fall over the red Sticker. We quickly realized that we would need to wheel inside the treadmill to stretch it enough to make it rotate. This fact caused trouble to determine if we needed one motor with a transmission gear or two separate motors for each wheel. Because we encountered technical difficulties with the gear (not solid enough), we decided to opt for the two motors solution.
Because of the shape of the motors, it was required to mount them vertically to not touch the treadmill.

This idea was abandoned because the structure was too fragile, would put too much weight on the back of the robot and was relatively inaccurate to drop cubes on the red sticker like shown in our tests.

C. Cage and Piston [Appendix F: Figure 7]

The cage and piston dispenser would consist of trapping the cubes in a cage, one top of each other. The cage would have a small opening at its bottom to let the first cube be pushed over by a piston. When the piston would retract, the second cube would fall, ready to be pushed again by the piston on a new red sticker.

The cage and piston system has been fully built and tested. It was concluded that although the structure was very stable and allowed a satisfying accuracy to “hit” the sticker with cube, we would observe on some occasions the cube not being dropped at all because of the too tight opening or a too wide cage. With the provided DPM kit material, it would have required more time to construct a more appropriate cage. Because our design team was running low on budget for the hardware development, we were forced to pass to an easier to develop dispenser.

D. Tennis Racket [Appendix F: Figure 8 and 9]

This design was based on simplicity. Our idea was to simply push over from a platform the cubes on the red stickers. We imagined a hand like or later a tennis like surface that would push progressively the first cube when a red sticker would be detected, then trigger a second time to push the second cube when desired.
The design was easy to build and show efficiency in the tests. The overall structure was reliable, accurate and did not provoke any inequal weight repartition to the robot.

When communicating with the sensor sub team (designer Yutong), it appeared that the color sensor would require a short distance from the ground. We adapted to this by hanging firmly the color sensor to the cube platform, in a way to avoid any collision between the color sensors and falling cubes that would induce inaccuracy.

We also précised that the robot would need to move back a bit as the color sensor would be exactly on top of the red sticker.

However, this dispenser design would require a precise placement of the cubes on the platform to assure that the cubes would not be moved by any of the robots' movements.
Whole system tests would falsify the preoccupation. It would be a human misplacement or mishandling during the robot setup that could move the cubes inappropriately.

Battery Holder:

A battery holder was necessary for the Brick Pi to operate. The battery being heavy and requiring being close to the Brick Pi because of the shortness of the alimentation wire, we decided to place the battery on top of the Brick Pi, holding it at several points.

Because of the additional weight of the battery, the robot would tilt backward and separate the wheels from the ground (impossible to move). Thus, two elements were added: a second steering ball at the back of the robot as well as a movable holder. The movable holder would allow us to extend it when we need to change or move the battery and close it when performing tests. [Appendix F: Figure 10]

Speaker/ Siren:

Looking at the requirements, the speaker could be placed anywhere at a close distance from the audio port of the Brick Pi. After some small test, it was confirmed that the Brick Pi could only be placed behind the dispenser racket. This design required adding some duct tape to fix the speaker to the Brick Pi. [Appendix F: Figure 11]

Brick Pi:

We had issues with the Brick Pi on the two kits we had.

- Brick Pi #29: the audio port wasn't working, this has been demonstrated during the labs
- Brick Pi #30: two motor ports (MA and MB) were not working either; this has been demonstrated during the tests across multiple subsystems such has wheel motors or dispenser.

The issue was resolved by getting a third Brick Pi at the DPM kit counter on Trottier 4th floor. At the end, we operated with Brick Pi #17.

Software Design:

The software was a central focus of our project, particularly the searching algorithm used in Room #2. The challenge was to develop a robust yet simple navigation method that worked reliably within the constraints of our hardware and budget.

Initial Approach

At first, we planned to use a Depth-First Search (DFS) algorithm to detect and extinguish fires in Room #2. This idea was promising from a theoretical perspective, but several factors led us to reconsider:

- Hardware limitations: Sensors and motors lacked the precision required for consistent DFS execution.
- Robot size: At approximately 15 cm, our robot struggled with tight maneuvers.
- Complexity: The software implementation was non-trivial and risky given the time and resources left.

Final Approach: Random Walk Strategy

Given these constraints and the remaining budget, we implemented a random walk algorithm inside Room #2. The behavior is simple but effective:

- The robot moves forward a short distance (3 cm), then scans using its ultrasonic and color sensors. It goes 45 degrees left and then 90 degrees right to sweep the floor with the color sensor.
- It continues straight unless one of the following conditions is met:
    - Red sticker (fire): Activates the dispenser and rotates randomly (45°–90°) after dropping a cube.
    - Green sticker (furniture) or another room: Reverses, rotates, and moves forward in a new direction.
    - Wall detected: Reverses, rotates, and changes path.

This loop continues until two fires are extinguished, after which the robot returns to base.
While this design doesn't guarantee completion within three minutes, it does guarantee eventual fire detection and extinguisher activation — a trade-off accepted due to tight resource constraints.

Return to Base Algorithm

To retrace its path, we implemented a stack-based movement logger:

- Every core displacement (excluding in-place scans) is pushed onto a stack.
- Once the fire mission is complete, the robot pops each movement off the stack in reverse and executes its inverse.
- This allows the robot to approximate its way back to position (0,0) and complete the mission.

Modular Design

Our code was modularized across multiple functions, with each team member responsible for one or more modules. The primary control was written in main.py, managed by Amine (Software Lead).

Table 4. Key functions

Function	Description
GoForward()	Moves the robot forward by a fixed distance
TurnLeft() / TurnRight()	Handles robot rotation at various angles
DetectWallDistance()	Uses ultrasonic sensor to detect obstacles
DetectColor() / ChooseColor()	Processes input from the color sensor
Dispenser() / ResetDispenser()	Activates and resets the fire extinguisher
PlaySiren()	Plays the 15-second siren during initial motion
IsTouchSensorPressed() / Emergency()	Emergency stop control
ReturnToBase()	Pops the movement stack and sends the robot home

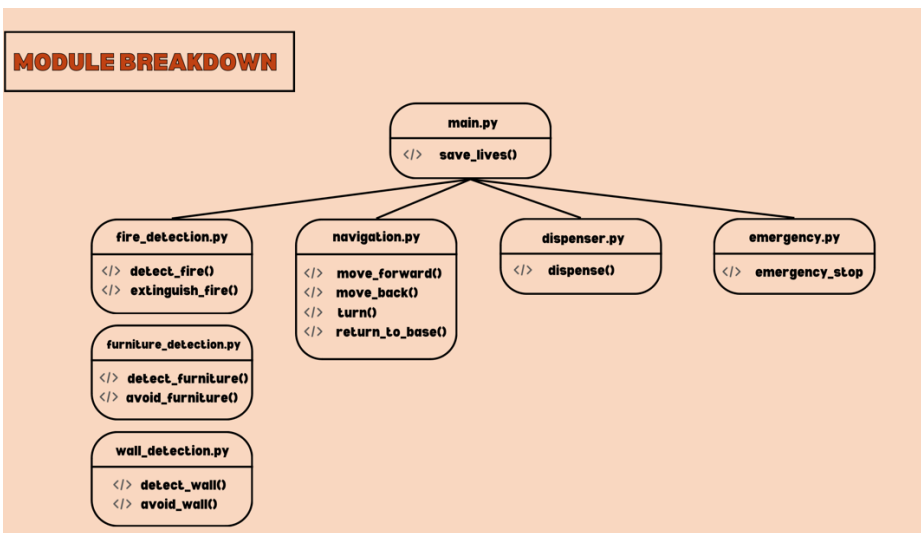


Figure 1.Breakdown of Modules

Recreating the System and Design Pitfalls to Avoid

Throughout the design process, we explored multiple hardware and system configurations that were ultimately discarded due to performance issues, complexity, or physical limitations. Below is a summary of these unsuccessful attempts and why future iterations should avoid repeating them:

1. Single Motor Navigation

- We tried: Using only one motor to drive the robot while relying on only two steering balls for balance.

- Why it didn’t work: The robot could not turn effectively with only one motor
- What to avoid: Always use two independently controlled motors for navigation. A single motor system lacks the rotation movement.

2. Mechanical Treadmill Dispenser

- We tried: A rotating belt mechanism powered by one or two motors to drop cubes.
- Why it didn’t work: The structure was too fragile and imprecise. Gear systems couldn’t generate enough tension, and vertical motor mounting caused instability.
- What to avoid: Avoid overly complex dispensers that require precise gear mechanics or stretchable elements.

3. Cage and Piston Dispenser

- What we tried: A piston-based system to push cubes out.
- Why it didn’t work: The system jammed frequently due to misalignment or inconsistent cube dimensions.
- What to avoid: Trying to implement a tight dropping mechanism.

4. Unbalanced Hardware Layout

- What we tried: Placing the battery directly on top of the BrickPi without extra support.
- Why it didn’t work: The robot tilted backwards under the battery's weight, lifting the front wheels off the ground.
- What to avoid: Avoid top-heavy designs without proper counterbalance. Add stabilizing elements like steering balls or weights to redistribute mass.

5. Overreliance on Pre-Defined Maps

- What we tried: Early in the process, we considered mapping room #2 in software to allow the robot to localize itself.
- Why it didn’t work: Hardware variability and movement inaccuracies made such a map unreliable. It added software complexity with limited benefits.
- What to avoid: Avoid hardcoded spatial assumptions unless localization is highly accurate. Dynamic sensor-based decision-making is more robust under uncertainty.

4. Testing

Dispenser Test

Table 5. Dispenser Iteration Process

Test	Date	Tester	Objective	Methodology	Conditions	Subsystems	Reference
1st Iteration: robotic arm	2025-03-15	-	Verify if it was possible to do it	Brainstorm and imagine if it could work	Paper Model	Dispenser	-
2nd Iteration: Mechanical treadmill	2025-03-28	Williams	Verify if the cubes touch the target	Build the model and test	Color sensor, displacement system, dispenser	Dispenser	Appendix G:Table 1
3rd Iteration: cage and piston	2025-03-28	Williams	Verify if the cubes touch the target	Build the model and test	Color sensor, displacement system, dispenser	Dispenser	Appendix G:Table 2
Last Iteration: Tennis Racket	2025-04-02	Hamza	Verify if the cubes touch the target	Build the model and test	Color sensor, displacement system, dispenser	Dispenser	Appendix G: Table 3

Detailed Description

We decided on 3 main criteria for the design testing:

- Accuracy (first and second block): Be sure to “hit” the red sticker with our cube
- Stability (of the blocks and the robot with the dispenser and the general dispenser structure): Not compromising the robot structure or loosing cubes at inappropriate timing.
- Reset Function (if applicable): Being able to put back the motors, cubes at their original placements easily.

We considered as satisfying a 90% accuracy or above, a 100% stability and reset function satisfying. Each technique was tested 10 times, all dispensers were posted in front of the red sticker, where they were the more accurate.
We didn’t test our first idea, we did not have a full overview on what are the capacities of the equipment we were provided with, so we sketched it out and realized that it was unfeasible without even testing it.

The test result for tennis racket showed a near perfect accuracy, good stability and easy implementation. This idea was selected for its performance. Also, after the tests, due to the

inconsistency, we decided to upgrade the motor used in the dispenser to one with a higher precision and better torque, ensuring consistent performance. This change will improve the overall accuracy, reducing delays in future releases. We also implemented a feedback loop to monitor motor speed and ensure it remains consistent.

Sensors Test

Color sensor testing

- Data collection

(Date: 13/03/2025, Tester: Yutong) V1.1
(Date: 15/03/2025, Tester: Yutong, Hamza) V2

1. Test objectives

The purpose of initial color testing is to analyze the color sensor's ability to distinguish between the different colors essential in detection, including both stickers (red, green), and background colors (yellow, purple, black, white, orange). The RGB values of specific colors were collected and analyzed under varying environmental conditions to establish reliable thresholds and criteria for accurate color classification.

2. Assumptions

It is assumed that the primary challenge in color classification lies in separating visually similar colors (e.g., red vs. yellow, green vs. black) under varying lighting and angle conditions. Rather than using a Euclidean distance approach in 3D RGB space, we chose to use ratio-based methods (such as R/G and R/B ratios), which provided a simpler and more interpretable classification strategy.

The Euclidean distance method, while mathematically robust, was not chosen due to the following assumptions:

- It weighs all three color-channels equally, which may dilute the specific distinguishing features (e.g., dominance of red or green) relevant to our color set.
- It is computationally more complex, which may introduce latency in real-time robot behavior.
- The ratio-based method showed clear visual separation in scatter and box plots, making it easier to define intuitive and data-driven thresholds.

- The task involves classifying a limited set of predefined colors, not identifying unknown colors, reducing the need for high-dimensional distance metrics.

Therefore, we assumed that a simpler, ratio-based classification strategy would be more efficient, and adaptable for the color-based navigation.

- Methodology & Data Processing

1. Filtering method

To reduce the effect of sensor noise and fluctuations caused by movement, lighting, or surface inconsistencies, a moving average filter was applied. The RGB readings were analyzed using a short moving average window (3 samples) to smooth out rapid variations without introducing delays in real-time detection. Additionally, extremely low values (e.g., R, G, B < 15) were flagged as potential black line or shadow interference and filtered out to prevent misclassification (when the color under detection is not black).

2. Processing techniques

The collected raw RGB values were further processed through statistical analysis of mean, minimum, and maximum values for each color under different conditions (normal, angled, shadowed). Moreover, calculations were made of R/G and R/B ratios to uncover consistent patterns unique to each color. Finally, percentile ranges were used to understand variability and support threshold decisions for classification.

3. Visualisation

To visualize color separation and verify trends:

- We generated scatter plots of Red vs. Green and Red vs. Blue values for each color.
- A linear regression line was fitted to the scatter plot of each color using software, allowing us to compute and compare slopes and intercepts.

These regression lines helped confirm that colors such as red exhibited direct proportionality, while others like green and purple had linear relationships with intercepts, reinforcing the value of ratio-based classification.

As Figure 2 shows, to determine relation of R and G value, the formula $R = kG + b$ is applied (where k, b are constants)

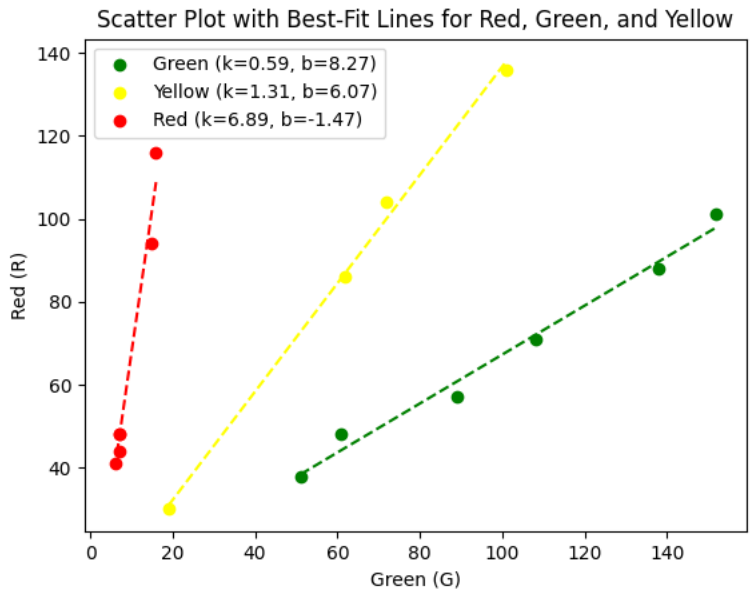


Figure 2. R vs. G ratio for Red, Yellow, and Green

As shown in Figure 2, the differences in slopes and intercepts in the R-to-G relationship indicate distinct color groupings. This suggests that a linear model could serve as a more precise classification criterion. However, since the intercept values are relatively small, their overall impact on classification is limited. Therefore, it may still be acceptable if we assume the relations are all directly proportional without the presence of intercepts.

This assumption is validated using box plots (calculating R/G ratios directly) as demonstrated in Appendix D: Color data analysis graphs This confirms the roughly acceptable range of the ratio and verifies it aligns with the value above. Following the similar procedure, the data for purple, orange and white is analyzed based on their range and R/B ratio.

<u>Detection software testing</u>

- Logic of the software

The classification logic is structured by prioritizing colors that are easier to detect and less likely to be confused (such as white and red) at the top of the decision tree. These colors typically exhibit strong, distinctive RGB patterns and are less sensitive to environmental variation.

As demonstrated in Appendix D, more ambiguous or overlapping colors, such as green, yellow, and purple, are evaluated later in the logic to avoid early misclassification. Similarly, background colors like black (shadows or grid lines) are filtered out early on based on brightness thresholds to reduce false positives.

Table 6. Final Classification Criteria Table

Color	**R Range**	**G Range**	**B Range**	**R/G, R/B Ratio**	**Decision Criteria**
Red	40 - 170	1 - 34	0 - 30	R/G ≥ 3.0	Low G, high R
Orange (ignored)	90 - 200	25 - 100	0 - 20	-	-
Green	0 - 125	40 - 186	4 - 26	R/G ≤ 1.0	High G, low R/G
Yellow	18 - 225	12 - 173	4 - 31	R/G ≥ 1.0	High G, high R/G
Black	0 - 20	0 - 20	0 – 20	-	Low RGB
white	70 - 225	70 - 225	70 - 225	-	High B
Purple	30 - 75	12 - 40	30 - 75	0.8< R/B < 1.5	R/B value

Ultrasonic sensor testing

To evaluate the ultrasonic sensor's accuracy and reliability, data was collected through continuous sampling of distance values in centimeters, as the sensor detected various surfaces at known distances. For each test, the measured values from the sensor were compared against actual physical distances (measured manually) to determine the level of error and variation under different environmental conditions and angles.

Based on the data collected, a detect_wall() function was or will be implemented to support wall or obstacle detection. This function will take a desired detection threshold as input (e.g., 5 cm), and return True if a wall or object is detected within that range, or False otherwise. This Boolean output can then be used by the robot to trigger avoidance behavior or adjust its navigation path. (The results are summarized in such way as demonstrated in Appendix E)

Navigation Test:

For the navigation, we divided the testing into three sections; From base to room #2, navigation in room #2 and return to base.

Test	**Test Date**	**Tester**	**Version**	**Objective**	**Methodology**	**Conditions**	**Subsystem**	**Reference**
From base to	2025-03-24	Justin Cyr	v1.0	Verify the Hardcode path	Programming test	Demo environment	Displacement System	Appendix I: Table

room #2								

Detailed Descriptions

We used a hardcoded path from base to room #2 by calculating the distances with a ruler and the angles that we want our robot to turn. Three criteria needed to be met for a success: enter the room #2, didn't go through a wrong room and passed by the door. At the end, the robot met them all by going forward, turn right, and turn left without colliding with a wall or entering a wrong room. Whenever a criterion was not met, we would adjust the functions in our code. For example, we added a correction factor cause the wheels were not turning at the same rate due to a weight disproportion. With a factor of 103/90, our robot would turn at the precise angles that we ask for. Then, the robot was able to systematically enter the room and respecting the requirements.

Test	Test Date	Tester	Version	Objective	Methodology	Conditions	Subsystem	Reference
Navigation into the room	2025-04-02	Amine	v1.0	Verify the robot moves correctly in room#2	Programming test	Demo environment	Displacement System, wheels, color sensor, UV sensor	Appendix J: Table

Detailed Description

For this test, we needed all the subsystems to work together. It was one of our last tests since we had to wait that the hardware, and all subsystems were fixed on the robot. For an overall success of the test the robot needed to stay in the room, avoid furniture and walls. Each time a criterion was not made, we adjusted the software.

Test	Test Date	Tester	Version	Objective	Methodology	Conditions	Subsystem	Reference
Return to Base	2025-04-05	Amine	v1.0	Verify the robot comeback to base precisely	Programming test	Demo Environment	Displacement System, wheels	Appendix K: Table

Detailed Description

To go back to base, our robot would follow the original path it did until room #2 but instead of going forward it would go backward. Correction factor whenever the robot failed the test. Again,

we used three criteria to verify it; Did it return to base? Did it enter an incorrect room? Did it want back by the door?

All the navigation tests were performed on the demo set because we realized that it was not moving the same distance in other settings.

Speaker Test:

Test	Test Date	Tester	Version	Objective	Methodology	Conditions	Subsystem and robot parts	Reference
Speaker Sound Test	2025-04-02	Hamza	v1.0	Verify Speaker functionality	Programming test	Standard user environment	Speaker, auxiliary cable, port	Appendix H: Table 1

Detailed Descriptions
One speaker was simply not working, and one Brick Pi had a failing audio port. This is why we decided to get a new Brick Pi at Trottier 4th floor. The test results with a working speaker are shown in appendix (see the reference above).
The speaker, if charged, was always displaying the firetruck Siren as desired during its route to room #2. The failure of the speaker led to a revision in the power distribution design to ensure that each component gets sufficient power and reduces the likelihood of such issues in the future. We also decided to use a more robust speaker model to minimize component failure risks.

Emergency Button Test:

The Emergency button had three different tests sets: hardcoded path phase, scanning phase, backtracking phase.

Test	Test Date	Tester	Version	Objective	Methodology	Conditions	Subsystem	Reference
Emergency Button	2025-04-08	Yutong	v1.0	Verify that the program and robot stop	Programming test	Standard user environment	Touch sensor	Appendix L: Table

Detailed description
With a chronometer, the tester would press the emergency button and calculate time also.
The emergency button would eventually work every time, but it required sometimes more than one touch by the user. This is probably due to a hardware issue (touch sensor) or due to a slow software. Also, we add trouble with the emergency button during the hardcode phase. Also, we

were at last minute for this test and that is why we didn't have time to change our model for a better one before demo day.

5. Performance

Performance

The robot's performance was evaluated based on the outlined specifications and operational scenarios provided by the users. Key findings regarding its performance are summarized below:

Sensor Stability and Accuracy

- The color sensor consistently maintained its position approximately 2 cm above the ground, meeting the required stability condition.
- A fixed tilt angle of 30 degrees to the vertical was successfully maintained, ensuring reliable color detection with minimal false readings.

Navigation Efficiency

- The ultrasonic sensors effectively detected walls and obstacles within the specified 4 cm range, ensuring near-zero collision incidents during navigation.
- Robot navigation into the fire room through the 22 cm-wide main entrance of Room 2 was precise, confirming the robot's reliability and maneuverability.

Fire Detection and Response

- The color sensor's detection range met the required minimum of 2 cm, consistently recognizing fires (red stickers) promptly.
- The sandbag dropping mechanism reliably activated within 0.3 seconds upon detecting fires, significantly contributing to mission efficiency.

Control and Responsiveness

- Robot turning speeds were effectively managed to remain within limits, ensuring accurate and timely color detection during operations.
- The emergency stop button was fully responsive, halting all robot movement immediately upon activation, ensuring safety and compliance with the user's emergency control specifications.

Mission Completion and Reliability

- The robot consistently started at the fire station (0, 0), completed its fire detection and extinguishing mission, and returned accurately to the station after task completion.

Limitations

While the robot performed well in its specified conditions, several limitations affect its overall functionality and performance:

Trade-Off Between Speed and Accuracy:

The robot executes a randomized navigation pattern that includes moving forward a fixed distance before initiating a sweeping motion. If the forward distance is too long, the robot may overshoot small target stickers and miss detection. While reducing the forward distance improves detection reliability, it results in slower overall movement, potentially making it unable to complete the task within the 3-minute time constraint. Thus, there is a trade-off between speed and precision that requires optimization.

Lighting and Sensor Alignment:

While the color classification is generally robust, accuracy can be affected by lighting variations and sensor misalignment. The robot does not adapt its path dynamically or map its environment, meaning it cannot respond to unexpected scenarios beyond its programmed conditions (such as encountering different colors).

Battery Limitations:

Low battery levels can slow down the robot's movements or deactivate the color sensor, affecting overall performance.

Inability to Handle Obstacles while moving:

The robot is not designed to handle fast-moving or dynamic obstacles. It operates under the assumption of static surroundings, limiting its response to simple, pre-programmed reactions. The robot also struggles in outdoor environments, where factors like light variations, uneven surfaces can interfere with sensor accuracy.

Narrow Field of View:

Due to its narrow scan path, the robot may miss detecting objects (e.g., stickers) if they fall outside this range. It does not include any self-correction or error recovery mechanism; once a target is missed, the robot will continue without reattempting detection.

Edge Cases:

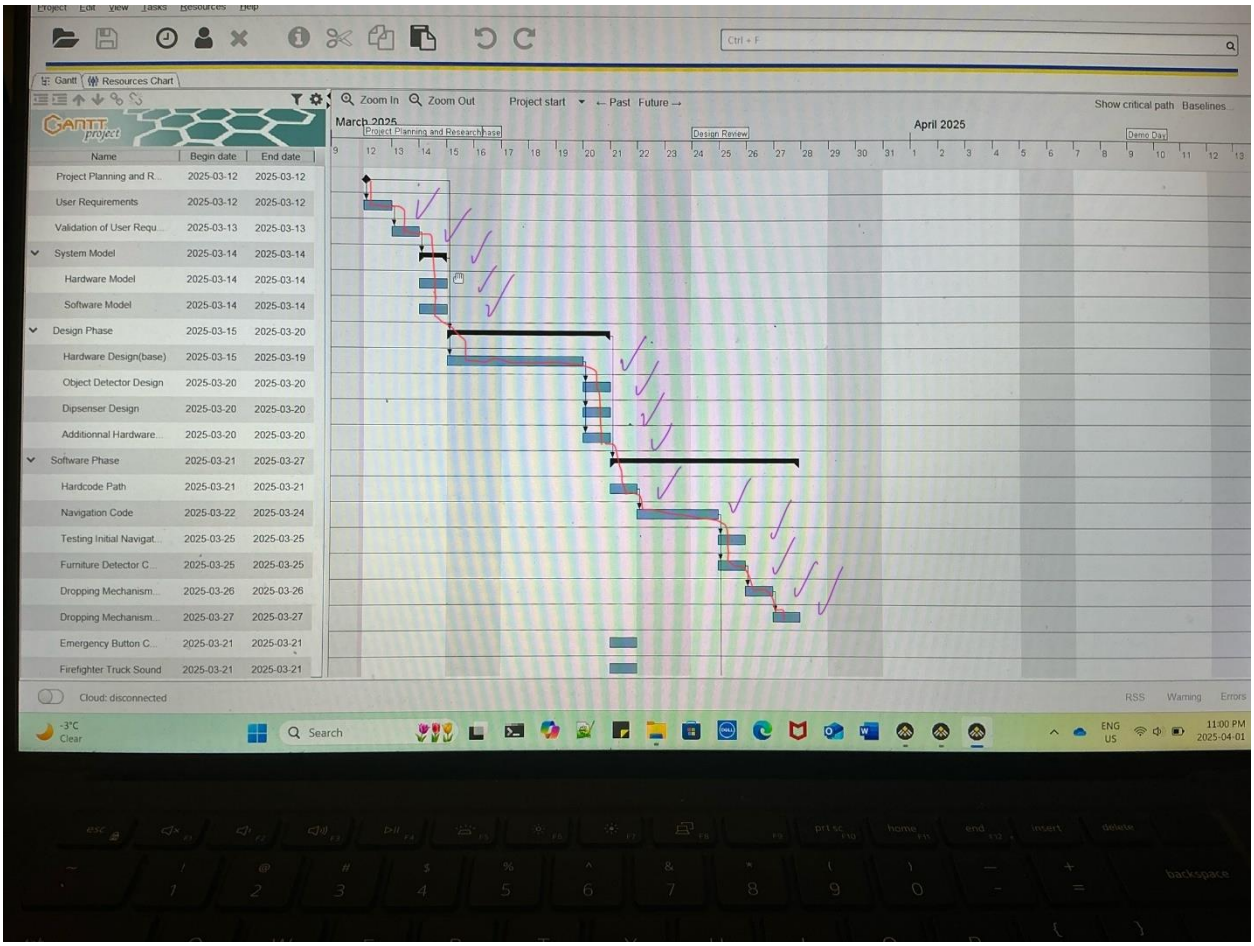
When fires or obstacles occur at the edge of the room (less than 5 cm from the wall), the robot may miss detection. The ultrasonic sensors will detect the wall and initiate turning functions to avoid collisions, which can lead to missed fire stickers in these areas.

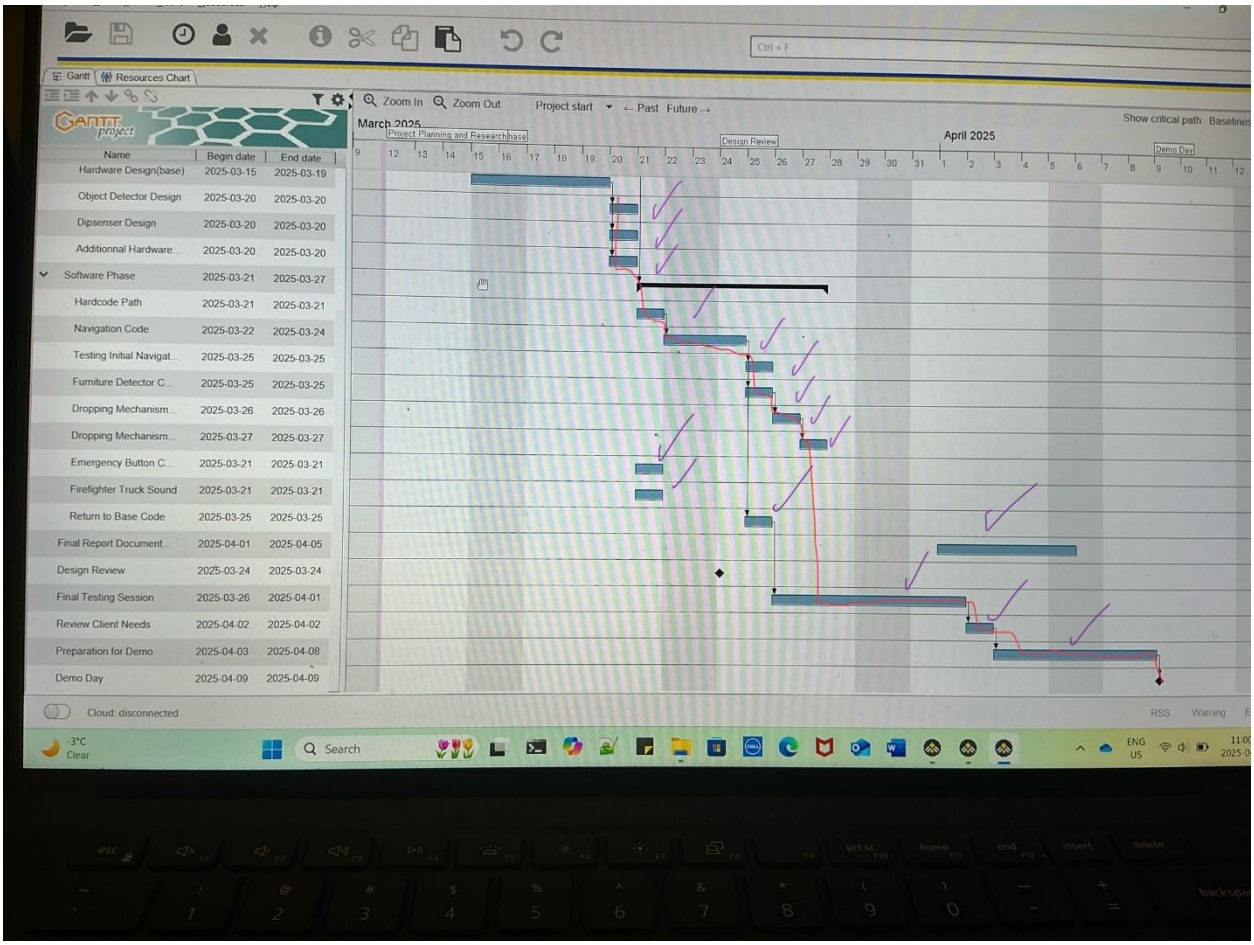
Given these limitations, the robot is best suited for controlled indoor environments such as lab experiments or classroom demonstrations on fixed-sized ground. It is not designed for real

firefighting scenarios or safety-critical applications. Users should avoid relying on the system in unpredictable or unstructured environments where adaptive flexibility is required.

6.Appendix

Appendix A: GanttChart

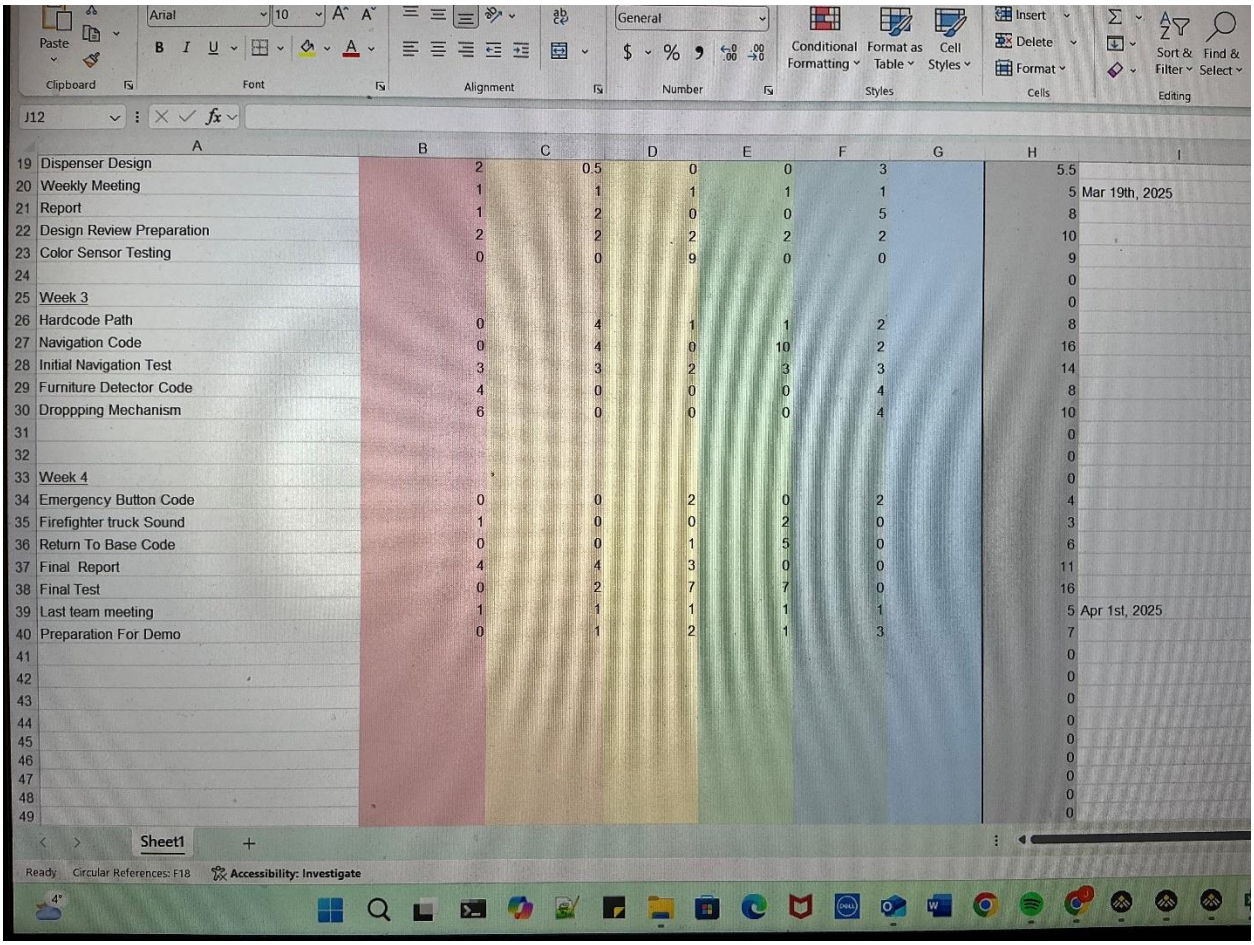




The critical path is shown in red, and the task completion is marked with purple marks. The Ganttchart document will be uploaded in a file. Every member was assigned to these different tasks, and it is shown in the task detailed resources which is available on the Ganttchart link.

Appendix B: TimeSheet

	Williams	Justin	Yutong	Amine	Hamza	Person 6	Total Hours	Date:
Total hours	36	36	36	36	36		180	
Week 1							0	
Project Planning and Research	0.5	0.5	0	0	0		1	
User Requirements	1.5	1.5	0	0	0		3	
GanttChart Creation	0	4	0	0	0		4	
Budget	2	0	0	0	0		2	
System Model(Hardware+ Software)	2	2	2	2	2		10	
Weekly Meeting(Validation of user requirements)	1	1	1	1	1		5	Mar 12th, 2025
Week 2							0	
Hardware Design	2	2	1	0	1		6	
Object Detector Design	1.5	0	1	0	0		3.5	
Dispenser Design	2	0.5	0	0	3		5.5	
Weekly Meeting	1	1	1	1	1		5	Mar 19th, 2025
Report	1	2	0	0	5		8	
Design Review Preparation	2	2	2	2	2		10	
Color Sensor Testing	0	0	9	0	0		9	
							0	
Week 3							0	
Hardcode Path	0	4	1	1	2		8	
Navigation Code	0	4	0	10	2		16	
Initial Navigation Test	3	3	2	3	3		14	
Furniture Detector Code	4	0	0	0	4		8	
Droppping Mechanism	6	0	0	0	4		10	
							0	



	A	B	C	D	E	F	G	H	I
19	Dispenser Design	2	0.5	0	0	3		5.5	
20	Weekly Meeting	1	1	1	1	1		5	Mar 19th, 2025
21	Report	1	2	0	0	5		8	
22	Design Review Preparation	2	2	2	2	2		10	
23	Color Sensor Testing	0	0	9	0	0		9	
24								0	
25	Week 3							0	
26	Hardcode Path	0	4	1	1	2		8	
27	Navigation Code	0	4	0	10	2		16	
28	Initial Navigation Test	3	3	2	3	3		14	
29	Furniture Detector Code	4	0	0	0	4		8	
30	Droppping Mechanism	6	0	0	0	4		10	
31								0	
32								0	
33	Week 4							0	
34	Emergency Button Code	0	0	2	0	2		4	
35	Firefighter truck Sound	1	0	0	2	0		3	
36	Return To Base Code	0	0	1	5	0		6	
37	Final Report	4	4	3	0	0		11	
38	Final Test	0	2	7	7	0		16	
39	Last team meeting	1	1	1	1	1		5	Apr 1st, 2025
40	Preparation For Demo	0	1	2	1	3		7	
41								0	
42								0	
43								0	
44								0	
45								0	
46								0	
47								0	
48								0	
49								0	

This is the original timesheet with hours each team member had done, we made sure that every team member spent there time equally in the lab.

Appendix C: Color Sensor Data Table

Table 7. Red color analysis

Trial Condition	**Mean (R, G, B)**	**Min (R, G, B)**	**Max (R, G, B)**
Angled 45°	(41, 6, 3)	(33, 1, 0)	(59, 11, 5)
Shadowed	(48, 7, 8)	(39, 5, 5)	(78, 11, 12)
Higher Position	(48, 7, 8)	(39, 5, 5)	(78, 11, 12)
Possible Shadows	(44, 7, 7)	(33, 4, 4)	(58, 16, 10)
Facing Light 1	(116, 16, 10)	(104, 12, 7)	(123, 20, 11)
Facing Light 2	(94, 15, 9)	(76, 10, 7)	(100, 17, 12)

V1.2

Table 8. Green color analysis

Trial Condition	**Mean (R, G, B)**	**Min (R, G, B)**	**Max (R, G, B)**
Outer Green	(48, 61, 15)	(33, 41, 10)	(60, 84, 19)
Green on Surface	(38, 51, 10)	(21, 23, 4)	(71, 110, 19)
Inner Green	(57, 89, 17)	(40, 49, 12)	(93, 138, 25)
Shadowed Green	(101, 152, 23)	(72, 104, 18)	(119, 186, 26)
Inner Facing Light	(88, 138, 21)	(63, 95, 16)	(109, 171, 25)
Outer Facing Light	(71, 108, 16)	(56, 84, 12)	(94, 142, 20)

Table 9. Yellow color analysis

Trial Condition	**Mean (R, G, B)**	**Min (R, G, B)**	**Max (R, G, B)**
(General)	(136, 101, 25)	(144, 105, 22)	(225, 173, 31)
(Darker Areas)	(30, 19, 7)	(18, 15, 4)	(64, 43, 12)
(Brighter Areas)	(104, 72, 18)	(89, 61, 14)	(125, 90, 20)
(Outer Areas)	(86, 62, 15)	(75, 47, 11)	(104, 76, 20)

Table 10. Purple color analysis

Trial	Mean (R, G, B)	Min (R, G, B)	Max (R, G, B)
1	(46, 23, 46)	(40, 19, 40)	(52, 26, 52)
2	(62, 33, 60)	(59, 29, 54)	(71, 37, 64)
3	(38, 17, 40)	(36, 16, 39)	(41, 19, 43)
4	(52, 25, 54)	(51, 23, 53)	(54, 27, 56)
5	(64, 33, 57)	(59, 30, 54)	(68, 38, 59)
6	(36, 17, 36)	(27, 13, 31)	(46, 22, 47)

Appendix D: Color data analysis graphs

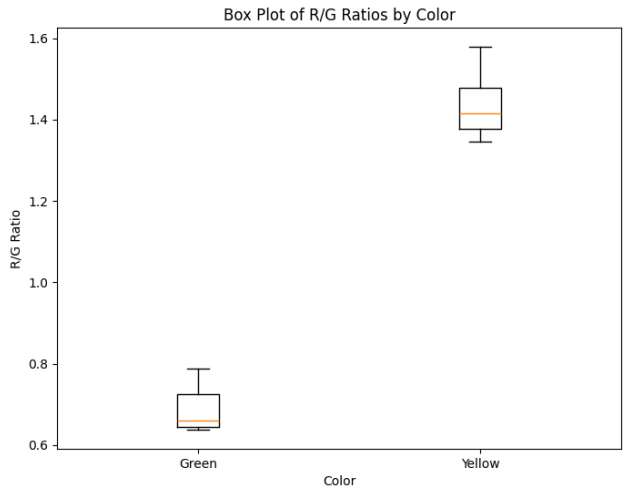


Figure 3. Example of box plots

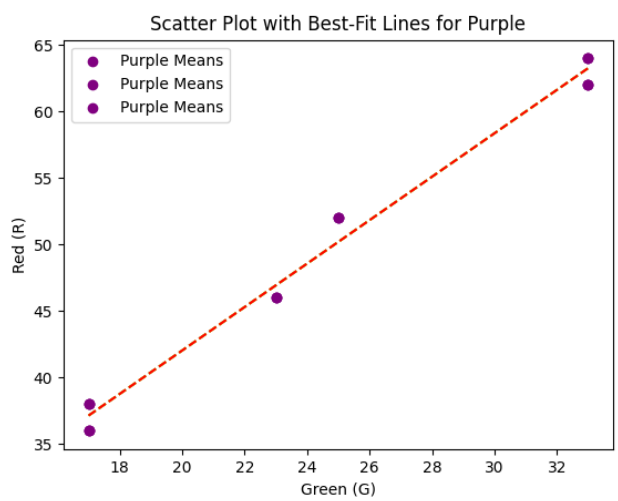


Figure 4. R vs. G (purple)

Appendix E: Ultrasonic Sensor Data

Table 11. "detect_wall" Function test (sample)

Test Number	#1	#2	#3	#4	#5	#6	#7	#8	#9
Distance from Wall	3 cm	3 cm	5cm	5 cm	5 cm	8cm	8cm	10cm	10cm
Distance returned by US sensor	3.05cm	3.04cm	5.06cm	5.03cm	5.02cm	7.96cm	8.02cm	10.0cm	10.04cm

Appendix F: Designs Photos and Sketches

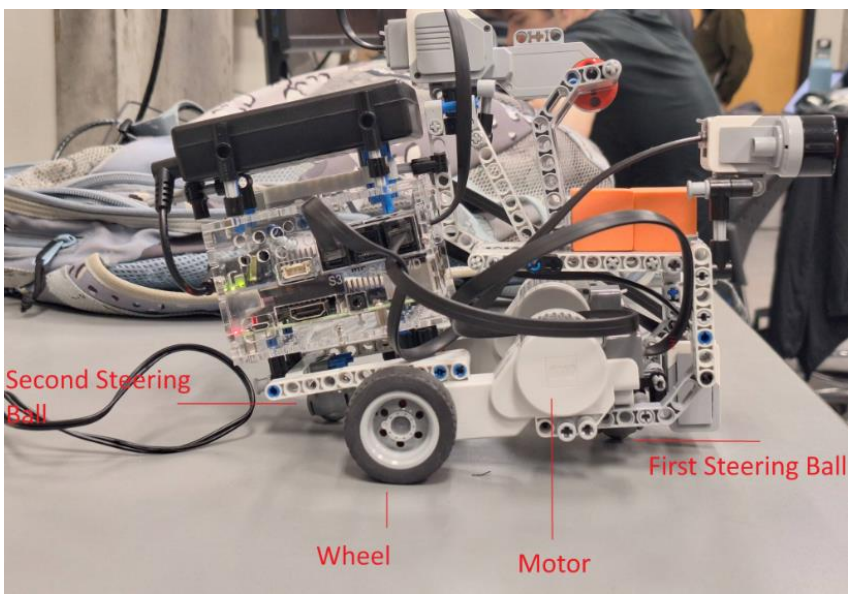


Figure 5. Prototype Side View: Displacement Subsystem

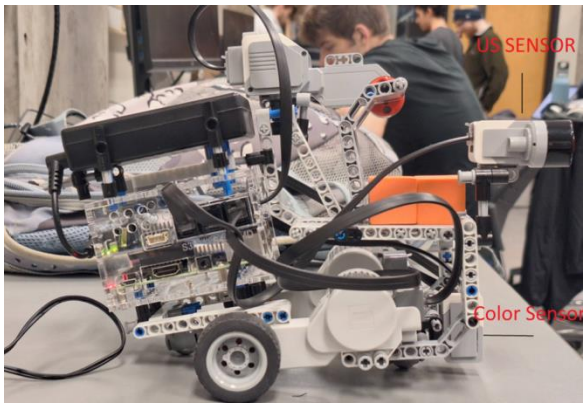


Figure 6. Prototype Side View: US and Color sensor

Figure 7. Prototype Top View: Touch Sensor, Emergency Button

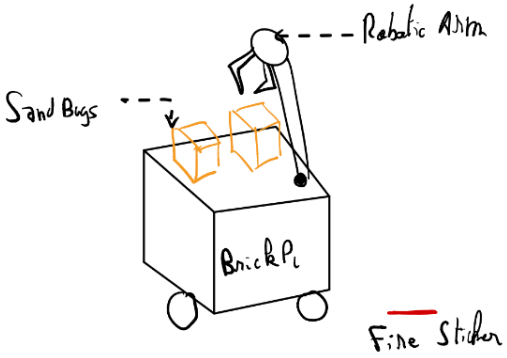


Figure 8. Dispenser Sketch: Robotic Arm

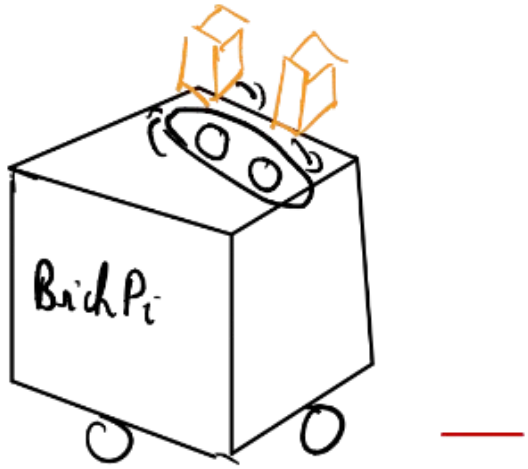


Figure 9. Dispenser Sketch: Mechanic Treadmill

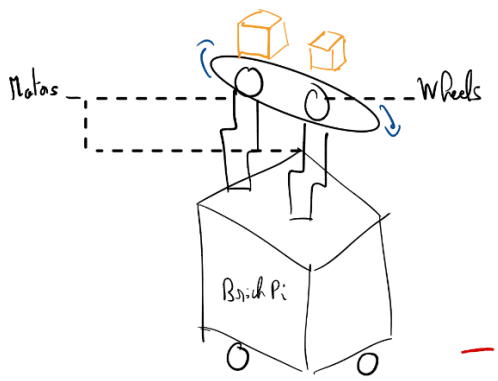


Figure 10. Dispenser Sketch: Mechanic Treadmill Structure

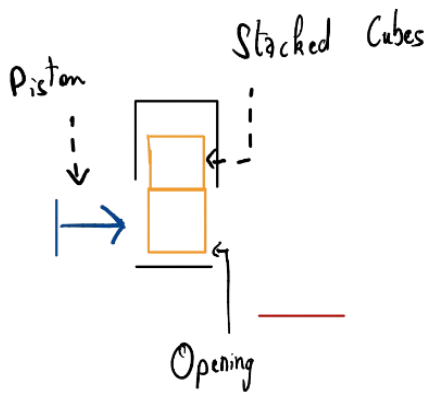


Figure 11. Dispenser Sketch: Cage and piston

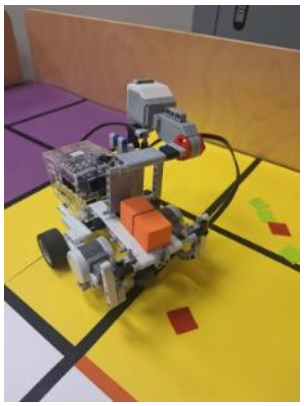


Figure 12. Prototype Photo: Tennis Racket

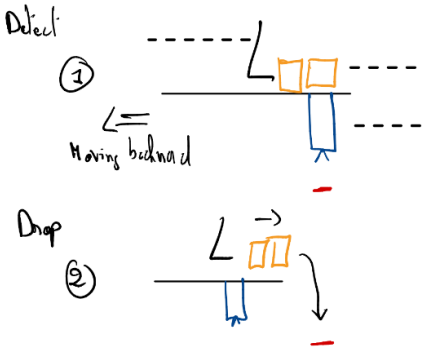


Figure 13. Dispenser Sketch: Dispenser Movements of Tennis Racket

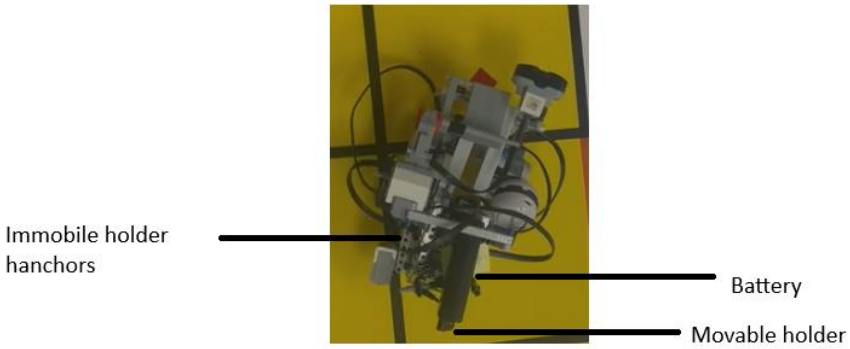


Figure 14. Top View of prototype: Battery Holder

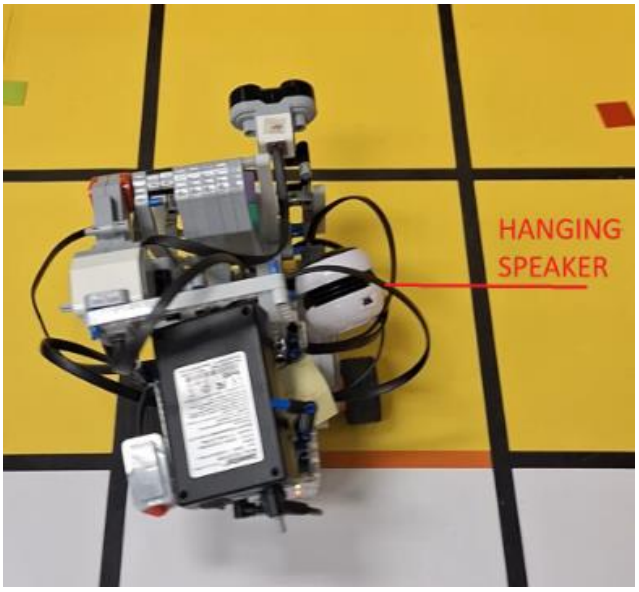


Figure 15. Top View of prototype: Hanging Tested Speaker

Appendix G: Dispenser Test Results

Table 12. mechanic Treadmill Test

Test Number	#1	#2	#3	#4	#5	#6	#7	#8	#9	#10	Results
First Cube touched target	No	No	Yes	No	No	No	Yes	No	Yes	Yes	40%
Second Cube touched target	Yes	Yes	Yes	No	No	No	Yes	Yes	No	Yes	60%
Stable	No	No	No	No	No	No	No	No	No	No	0%
Was easy to replace	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	100%

Table 13. Cage and Piston

Test Number	#1	#2	#3	#4	#5	#6	#7	#8	#9	#10	Results
First Cube touched target	Yes	No	Yes	No	Yes	Yes	Yes	Yes	Yes	Yes	80%
Second Cube touched target	Yes	No	No	No	Yes	Yes	Yes	Yes	No	Yes	60%
Stable	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	100%
Was easy to replace	No	No	No	No	No	No	No	No	No	No	0%

Table 14. Tennis Racket

Test Number	#1	#2	#3	#4	#5	#6	#7	#8	#9	#10	Results
First Cube touched target	Yes	Yes	Yes	No	Yes	Yes	Yes	No	Yes	Yes	90%

Second Cube touched target	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	100%
Stable	Yes	Yes	Yes	No	Yes	Yes	Yes	Yes	Yes	Yes	90%
Was easy to replace	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	100%

Appendix H: Speaker and Emergency Button

Table 15. Speaker Testing results

Test Number	#1	#2	#3	#4	#5
Was the Speaker Displaying the Siren?	Yes	Yes	Yes	Yes	Yes

Appendix I: Displacement for hardcoded path

Table 16. Hardcoded path test results

Test Number	#1	#2	#3	#4	#5	#6	#7	#8	#9	#10
Did the robot enter the kitchen?	No	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
Did the robot collide a wall, entered a wrong room?	No	Yes	Yes	Yes	No	No	No	No	No	No

Did the robot use the kitchen's door?	No	No	No	Yes	Yes	Yes	Yes	Yes	Yes	Yes

Appendix J: Navigation into the room test

Table 17. Navigation Tests Results(room)

Test Number	#1	#2	#3	#4	#5	#6	#7	#8	#9	#10
Did the robot came avoid furniture's	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
Did the robot collide a wall	Yes	Yes	No	Yes	No	No	No	No	No	No
Did the robot entered a wrong room?	Yes (purple)	Yes(purple)	No	Yes(White)	No	No	No	No	No	No

Appendix K: Return to Base Test

Table 18. Navigation Tests Results (Return to base)

Test Number	#1	#2	#3	#4	#5	#6	#7	#8	#9	#10
Did the robot came back by the door	No	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
Did the robot collide a wall, entered a wrong room?	No	Yes	Yes	Yes	No	No	No	No	No	No

Did the robot use came back to base?	No	No	No	Yes	Yes	Yes	Yes	Yes	Yes	Yes

Appendix L: Emergency Button

Table 19. Emergency button, Scanning phase

Test Number	#1	#2	#3	#4	#5
Time Before Stopping (in seconds)	Did not stopped	3.0s	2.5s	1.5s	2.6s
Number of time touch sensor was pressed	5	5	4	2	4

Table 20. Emergency button, Hardcode path phase

Test Number	#1	#2	#3	#4	#5
Time Before Stopping (in seconds)	Did not stopped	Did not stopped	Did not stopped	Did not stopped	Did not stopped
Number of time touch sensor was pressed	-	-	-	-	-

Table 21. Emergency button, Backtracking phase

Test Number	#1	#2	#3	#4	#5
Time Before Stopping (in seconds)	2.9s	3.2s	1.2s	3.0s	2.5s
Number of time touch	2	5	1	4	4

sensor was pressed					